# Descomposiciones en triángulos I: Documento de formalización y reproducibilidad

Juan Pablo Traverso Gianini
Investigador independiente, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

**Documento complementario del Paper I de la serie:** *Descomposiciones en triángulos I: Formalización del teorema de Nash-Williams en Lean* [1], preprint 1.0 del manuscrito.

**Paper II de la serie:** *Descomposiciones en triángulos II: Absorción robusta para triángulos tipados* [2].

**Preprint 1.0 — 20 de octubre de 2026.**

## Resumen

Este documento registra el alcance formal y la evidencia de reproducibilidad de Paper I. El archivo seleccionado conserva la evidencia de compilación limpia de 1.070 módulos del proyecto y seis consultas finales de axiomas, y corrige las cabeceras de autoría de sus fuentes. Un registro reversible de huellas relaciona las fuentes corregidas con los bytes compilados; no se afirma un nuevo build de Lean. Identificamos el teorema final, los resultados fraccionales, las entradas tipadas de sustitución, el objetivo exacto de auditoría, la alineación de fuentes y las dependencias fijadas. Distinguimos esta compilación local de una reproducción independiente y de la validación separada de Paper II. La presente actualización editorial inspecciona la evidencia entregada sin ejecutar Lean de nuevo.

## 1. Alcance e identidad de las fuentes

Paper I formaliza el teorema de Delcourt y Postle [3, Teorema 1.8]: todo grafo suficientemente grande, divisible en triángulos y con grado mínimo al menos igual a tres cuartos de su orden admite una descomposición en triángulos. El desarrollo formal incluye también su teorema fraccional y sus resultados de estabilidad. Este documento identifica las declaraciones, los resultados de sustitución usados en la prueba entera y la evidencia correspondiente a una versión fija de las fuentes. Es un registro técnico, no un reemplazo de las demostraciones matemáticas de Papers I y II.

La compilación limpia histórica, anterior a la corrección de cabeceras, terminó satisfactoriamente. Su objetivo explícito fue `RequestProject.NW18.AuditPaperIIAssembly`, y el teorema final es `NashWilliams18.PaperI.nash_williams_via_paperII`. Los 1.070 módulos del proyecto en la clausura de importaciones tienen recibos satisfactorios. Las huellas de sus fuentes y de los objetos importados del proyecto forman una cadena coherente. Pasaron las seis consultas finales de axiomas y la auditoría de dependencias de la prueba.

Estas afirmaciones se refieren a ese objetivo, no a todos los archivos incluidos ni al árbol actual completo de Paper II. Paper I utiliza los resultados particulares de Paper II presentes en esta versión congelada. La validación incremental separada de Paper II registrada en `PAPER_II_`

`LEAN_20261019_v1` pasó, incluidas las diecisiete familias y la exhaustividad del censo. No es necesaria para certificar la versión congelada de Paper I.

El freeze de fuentes seleccionado es `PAPER_I_LEAN_20261020_ALIGNED_v3`. El SHA-256 del archivo es

`bfa42bd57e6d9c2ac918bbf6fe2c2ba7c01894d1d8387d92e81b090020a6aaff`.

Preprint 1.0 es la versión de publicación de este documento técnico dentro de la serie. Los documentos anteriores de revisión privada y el companion conservado dentro del archivo original mantienen sus propias etiquetas de versión. Este documento no reemplaza esos registros ni modifica el archivo. Sus metadatos de lanzamiento y sus huellas documentales se registran por separado.

El registro de build conservado describe la compilación desde un directorio de salida del proyecto vacío, sin reutilizar objetos del proyecto de builds anteriores. El anterior `PAPER_I_LEAN_20261017_v1` se conserva como evidencia histórica; su pasada incremental final recompiló cinco módulos. Ninguno de los dos registros es una reproducción independiente por terceros. La Sección 6 describe las reanudaciones dentro de la campaña limpia y el uso de productos de build de Mathlib en caché.

## 2. Enunciado matemático y definiciones

La declaración principal comprobada tiene el orden de cuantificadores

$$\begin{gathered}\exists N \in \mathbb{N}\ \forall G : \\ n \geq N, \quad 3 \mid |E(G)|, \quad (\forall v,\ 2 \mid d_G(v)), \quad 3n \leq 4\delta(G) \\ \implies \quad G \text{ admite una descomposición en triángulos.}\end{gathered}$$

Aquí $G$ es un grafo simple finito, $n = |V(G)|$, y el umbral de orden se elige antes del grafo. El umbral de grado es exactamente $3n/4$, no $(3/4 + \varepsilon)n$. La declaración afirma que existe $N$; este documento no proporciona un valor numérico.

`NashWilliams.IsTriangleDivisible` exige que `G.edgeFinset.card` sea divisible por tres y que `Even (G.degree v)` valga para todo vértice. `NashWilliams.IsTriangleDecomposition` exige una familia finita de cliques de tres elementos y un único miembro que contenga los extremos de cada arista. `NashWilliams.HasTriangleDecomposition` cuantifica existencialmente esa familia. Las definiciones están en `RequestProject/NashWilliamsBasic.lean`.

Para comparar con el enunciado matemático, el tipo con los espacios de nombres abiertos es:

```
Exists fun N : Nat =>
  forall (V : Type) [Fintype V] [DecidableEq V]
    (G : SimpleGraph V) [DecidableRel G.Adj],
    N <= Fintype.card V ->
    IsTriangleDivisible G ->
    3 * Fintype.card V <= 4 * G.minDegree ->
    HasTriangleDecomposition G
```

El enunciado formal lleva además las instancias de finitud y decidibilidad necesarias para representar un grafo finito en Lean. Son parámetros de representación, no hipótesis adicionales de teoría de grafos. Ni una descomposición fraccional ni un certificado de absorción son premisas de la declaración final.

## 3. Enunciados disponibles en la clausura seleccionada

Las rutas siguientes son relativas a `source/` dentro del archivo. Todos los módulos enumerados pertenecen a la clausura de importaciones comprobada. Su inclusión aquí no significa que cada declaración haya recibido una consulta final de axiomas por separado: la Sección 5 enumera exactamente las seis declaraciones consultadas.

**Umbral fraccional, Delcourt-Postle 1.4.** `NashWilliams.Frac.exists_fractional_weights`, en `RequestProject/Fractional/Final.lean`, produce pesos reales no negativos sobre conjuntos finitos de vértices. Para cada arista, la suma sobre las cliques de tres vértices que contienen sus extremos es uno. La única desigualdad sobre el grafo es `3 * Fintype.card V ≤ 4 * G.minDegree`; no exige divisibilidad entera ni orden suficientemente grande.

**Estabilidad fraccional, Delcourt-Postle 1.6.** `NashWilliams.Frac.theorem_1_6_proof`, en `RequestProject/FracStab/Main.lean`, elige primero $\exists \sigma_0 > 0$, después $\forall \sigma \in (0, \sigma_0]$ y luego $\exists \varepsilon > 0$ y $n_0$. Por encima de ese orden y del grado $(3/4 - \varepsilon)n$, la inexistencia de una descomposición fraccional en triángulos implica `IsExtremal σ G`.

**Estabilidad fraccional regular, Delcourt-Postle 2.6.** `NashWilliams18.theorem_2_6`, en `RequestProject/NW18/Theorem26Proof.lean`, elige también $C \geq 1$ antes del grafo. La inexistencia de una descomposición fraccional `C`-regular implica la misma conclusión de extremalidad. La descomposición regular incluye cobertura exacta de las aristas, no solamente aproximada.

**Teorema entero, Delcourt-Postle 1.8.** `NashWilliams18.PaperI.nash_williams_via_paperII`, en `RequestProject/NW18/PaperIIAssembly.lean`, es el teorema final enunciado en la Sección 2 de este documento y en (1.1) del manuscrito. La declaración intermedia `NashWilliams18.PaperI.nash_williams_of_paperII_inputs` todavía recibe `Branch124Statement` y `Theorem146Statement`; la declaración final proporciona ambas pruebas. La distinción importa: un adaptador con premisas pendientes no es el resultado completo.

**Resultado de emparejamientos de Ehard-Glock-Joos.** La declaración local utilizada es `Pseudorandom.theorem_1_2`. Su presencia aquí no afirma que el objetivo de Paper I haya auditado todos los teoremas de aquel artículo. La formalización separada de EGJ tiene su propio registro y espacio de nombres; el manuscrito la documenta por separado. Durante este freeze no se sustituyó la fuente local por un paquete público.

## 4. Las cuatro ramas de sustitución

Delcourt y Postle [3] usan el Teorema 13.16 y atribuyen su prueba al manuscrito que figura como referencia 17. Al preparar Paper I no se había localizado una versión pública de ese manuscrito. Lo que afirmamos aquí es que nuestra prueba seleccionada no depende del Teorema 13.16. No inferimos que el manuscrito no exista, que su teorema sea falso ni que nuestro resultado equivalga a un enunciado no disponible.

| Rama de [3] | Sustitución en el ensamblaje comprobado |
|---|---|
| 12.4, no extremal | El productor demostrado de la ronda protegida, seguido de la construcción terminal de absorción. |
| 12.5 mediante 14.6 | El teorema semidenso separado `F0Direct.theorem146_ordered`, mediante el adaptador F0. |
| 12.6 y 12.7 mediante 14.7 | El resultado tipado casi completo `PaddedGeneral.RealInverse.idealized_F12`, mediante los adaptadores F1 y F2. |
| 12.8, cuatro colores | El nuevo puente F3 hacia `PaddedGeneral.RealInverse.nearlyComplete_F123`. |

La entrada no extremal es `roundUnivSpec124_holds`, que se aplica mediante `roundSpec124_of_univ` y `branch124_of_round`. Se construye en el árbol congelado, no se postula. Esta rama no es una aplicación de blowups tipados casi completos.

El adaptador F0 es `NashWilliams18.PaperII.theorem146_F0`. Su prueba usa el teorema semidenso separado; el ensamblaje no atribuye F0 a la geometría de F1 y F2. La implementación cuenta discrepancias ordenadas: cada arista editada no ordenada contribuye dos. El umbral del manuscrito corresponde, por tanto, al doble de discrepancias ordenadas, como se indica en Paper II, Sección 5. Los adaptadores conservan esta conversión.

Los adaptadores F1 y F2 son `NashWilliams18.PaperII.theorem147_F1` y `NashWilliams18.PaperII.theorem147_F2`. En `BlowupPaperInputs.lean`, `idealized_iff_paperII` se demuestra mediante `Iff.rfl`: los predicados del anfitrión son idénticos por definición. Es una identificación explícita de hipótesis, no una conversión inferida por semejanza de nombres.

El puente F3 es `NashWilliams18.PaperII.margin_four_colour_decomposition`, en `PartitionedTriangleAbsorption/PaperII/FourColourApplication.lean`. Recibe una coloración propia de cuatro colores, la condición de grado próxima al umbral, grados pares y los dos balances de conteos de aristas entre pares opuestos. Su conclusión proporciona una descomposición F3 y la descomposición F0 obtenida al agrupar colores, sobre el mismo conjunto de aristas. Elige

$$\varepsilon = \min\{1/16, \eta/3, \delta/4\}.$$

Las cotas de tamaños de clases dan un grado de aristas faltantes a lo sumo $4\varepsilon n$; están demostradas las dos desigualdades necesarias para el piso que se usa como parámetro entero. La auditoría final exige este puente y prohíbe el antiguo `FourColourConsumerStage84.margin_four_colour_decomposition` en la clausura de dependencias de la prueba del teorema.

En el freeze separado `PAPER_II_LEAN_20261019_v1`, la entrada final `PaperIApplications` de Paper II consulta `PartitionedTriangleAbsorption.FourColourConsumerStage84.margin_four_colour_decomposition`. Paper I excluye ese consumidor anterior de su prueba final y utiliza en cambio `NashWilliams18.PaperII.margin_four_colour_decomposition`, que invoca `nearlyComplete_F123`. Este último tiene evidencia de axiomas a nivel de módulo en Paper II, pero no es una de sus quince consultas finales de entrada. La auditoría de dependencias y las consultas del teorema final de Paper I certifican la ruta realmente utilizada por Paper I. No deben describirse objetivos consultados distintos como un único punto de entrada sincronizado. El archivo alineado de Paper I certifica el puente trasladado mediante su propio objetivo final. No certifica una nueva auditoría de `PaperIApplications` para Paper II; eso requiere el freeze separado de Paper II.

## 5. Evidencia de axiomas y dependencias

`AuditPaperIIAssembly.lean` recorre las constantes presentes en los tipos y términos de prueba del teorema final. Comprueba las sustituciones exigidas y excluye la antigua ruta global. Esto es más preciso que encontrar una importación en una fuente: una declaración importada puede estar disponible sin intervenir en la prueba final.

Las seis consultas finales son:

1. `NashWilliams18.PaperII.theorem146_F0`;
2. `NashWilliams18.PaperII.theorem147_F1`;
3. `NashWilliams18.PaperII.theorem147_F2`;
4. `NashWilliams18.PaperII.typed_inputs`;

5. NashWilliams18.PaperI.nash_williams_of_paperII_inputs;
6. NashWilliams18.PaperI.nash_williams_via_paperII.

Cada una informó exactamente propext, Classical.choice y Quot.sound. Ninguna informó sorryAx ni un axioma matemático adicional. Son los axiomas fundacionales de Lean, por lo que sería incorrecto escribir «sin axiomas». La auditoría final no contiene errores ni advertencias. Los registros de compilación de módulos sí contienen advertencias; no se afirma que el proyecto esté libre de ellas.

La auditoría de dependencias informa PASS_PAPERII_PROOF_DEPENDENCIES. Entre las constantes exigidas están el teorema F0, idealized_F12, el puente F3, nearlyComplete_F123, completeInverseCertLocalBound_general y UniversalProducer.producerTarget_of_threeStepLinks. Entre las prohibidas están NashWilliams18.Theorem1316Statement, la antigua declaración nash_williams, la ruta alternativa de vórtice F1 y el antiguo consumidor F3. Las listas completas están en la fuente congelada de la auditoría.

El sello del archivo v3 reporta PASS_HEADER_ONLY_REVISION. Sus 1.070 cabeceras de fuentes identifican a Juan Pablo Traverso Gianini. La evidencia histórica de compilación limpia no cambia y corresponde a las fuentes reconstruidas anteriores a la corrección, no a las huellas actuales. El archivo interno evidence/verification.json tiene SHA-256

d83783e3a0dbcc7d3bf1a465460b3996c0889dc6371d362d766e0e48f6583497.

Esa huella identifica el registro sellado de verificación, no el ZIP ni este documento. Sus huellas se registran por separado.

## 6. Identidad de las fuentes, paquetes y alcance de compilación

El toolchain es leanprover/lean4:v4.35.0-rc3. Mathlib está fijado en c55e6e786f49471c72fbddbec5415808896aec1e; el archivo de dependencias fija ocho paquetes auxiliares. Sus revisiones figuran en evidence/environment.json y source/lake-manifest.json. La compilación utilizó los productos de build de esa revisión de Mathlib. No se recompiló Mathlib.

La clausura seleccionada tiene 1.070 módulos: 251 en RequestProject y 819 en PartitionedTriangleAbsorption, incluido su módulo raíz. El archivo contiene exactamente esas fuentes del proyecto. No certifica el desarrollo completo de Paper II.

Antes de la revisión de cabeceras, la alineación parte de los freezes anteriores de Paper I y Paper II. Los 623 módulos previamente compartidos toman la fuente de Paper II. Así se resuelven las cuatro diferencias en TypedRefinerF0.PermCount, TypedRefinerF0.Estimates, TypedRefinerF0.RobustHall y BlowupB.PaddedGeneral.RealInverse.RoutedPaper. Los 194 módulos de ese espacio de nombres exclusivos de Paper I conservan sus fuentes. La aplicación de cuatro colores se traslada, sin modificar su contenido ni los nombres de sus declaraciones, de RequestProject.NW18.FourColourPaperII a PartitionedTriangleAbsorption.PaperII.FourColourApplication; el import de Case128Reduction sigue ese traslado. La clausura resultante tiene un módulo adicional. El manifiesto de alineación registra el origen de las fuentes y los cambios; no afirma que el objetivo de Paper I haya compilado todos los módulos del desarrollo alineado completo.

La compilación limpia comenzó con un directorio de salida del proyecto vacío. Se realizó en varias invocaciones de tools/build.py paper_I, reanudadas tras ajustes de recursos. Las invocaciones reanudadas solo aceptaron objetos con recibos satisfactorios de esta campaña y hashes vigentes de fuente y objeto. No se reutilizaron objetos del proyecto de los freezes anteriores. El objetivo final compiló las seis consultas y la comprobación de dependencias de la prueba.

Al sellar el paquete, tools/make_freeze.py comprobó los 1.070 recibos, las huellas de fuentes y

objetos y cada huella registrada de los objetos importados del proyecto. Los primeros 23 módulos habían utilizado inicialmente otro checkout de Mathlib de la misma revisión; al recompilarlos contra el checkout final, los 23 objetos resultaron idénticos byte por byte. Es una compilación limpia local completa, no una reproducción independiente por terceros.

## 7. Cómo reproducir el objetivo seleccionado

El archivo conserva los scripts utilizados en la campaña. Estos emplean inspección de memoria específica de Windows y rutas fijas de trabajo; no son un adaptador portable que pueda ejecutarse desde cualquier directorio de extracción. Se omiten los ejecutables de Lean, los objetos compilados del proyecto y las cachés de dependencias.

Para repetir el procedimiento registrado, primero deben inspeccionarse `tools/align.py` y `tools/build.py` y prepararse ubicaciones aisladas para sus rutas de archivos, fuentes y resultados. Se necesitan los dos archivos de fuentes identificados en `evidence/environment.json`, cuyas huellas deben verificarse, y el toolchain fijado. `align.py` reconstruye el árbol alineado y el plan por fases a partir de esos archivos. Deben proporcionarse Mathlib y los ocho paquetes auxiliares en las revisiones fijadas, con sus productos de build correspondientes, y comprobar esas revisiones antes de invocar el compilador dirigido.

Las siguientes operaciones reproducen la alineación histórica. Para compilar la edición corregida, debe utilizarse su árbol `source/` con las dependencias fijadas y recibos nuevos; no debe sobrescribirse mediante el script histórico de alineación. Las operaciones de la campaña son:

```
elan toolchain install leanprover/lean4:v4.35.0-rc3
python tools/align.py
python tools/build.py paper_I --workers 4 --threads 3 --memory-mb 5000
```

Estos comandos requieren la preparación de rutas y dependencias descrita arriba. La primera invocación debe utilizar un directorio de salida del proyecto vacío. Una continuación de la misma campaña puede usar `--resume`; esa opción no sustituye el inicio limpio. `tools/make_freeze.py` verifica la cadena de huellas de fuentes, objetos e imports y prepara un registro nuevo. Debe ejecutarse con una ubicación de salida separada, sin sobrescribir el archivo seleccionado.

La evidencia histórica puede inspeccionarse sin Lean ejecutando `python tools/verify_attribution.py` desde la raíz del archivo extraído. El script verifica las huellas actuales y utiliza `attribution/source_changes.json` para reconstruir cada fuente exacta anterior a la corrección a partir de su cabecera original y del cuerpo conservado. Compara esas huellas reconstruidas con los 1.070 recibos históricos y comprueba los 1.671 enlaces de huellas de imports del proyecto y el manifiesto interno. Los objetos compilados se omiten, por lo que sus bytes no pueden comprobarse de nuevo solo a partir del archivo; el productor los comprobó durante el sellado original. No se afirma una nueva compilación ni identidad de objetos para las huellas de fuentes corregidas.

## 8. Cómo leer el archivo

`evidence/ALIGNMENT_MANIFEST.json` identifica el origen de las fuentes y los cambios de alineación. `evidence/build_plan.json` da el objetivo, el orden topológico de sus 1.070 módulos y los imports del proyecto. `evidence/receipts/` contiene huellas de fuentes, dependencias y objetos; `evidence/module_logs/` conserva la salida del compilador para cada módulo. `evidence/final_audit.log` contiene las seis consultas finales y el indicador de éxito de dependencias. `evidence/verification.json` registra las comprobaciones del sellado; `evidence/early_module_recheck.txt` registra la comparación de 23 módulos.

`source/` contiene la clausura seleccionada y la configuración fijada; `tools/` conserva los scripts

de la campaña. `SHA256SUMS.txt` vincula 3.240 archivos internos. El ZIP contiene 3.241 archivos contando ese manifiesto, y su archivo de huella externo da el resumen del ZIP. Los manuscritos y documentos complementarios se mantienen fuera de este archivo Lean.

La versión v3 añade 1.008 cabeceras de autoría faltantes, corrige 59 casos del apellido y sustituye una atribución a una herramienta computacional por Juan Pablo Traverso Gianini. Dos cabeceras ya eran correctas. Los 1.070 archivos identifican ahora a ese autor y Apache-2.0. Se conserva cada byte posterior al comentario ordinario inicial de cabecera, incluidas las declaraciones, las pruebas y los comentarios de procedencia. No cambian los nueve textos de licencias de dependencias ni los 2.147 archivos preexistentes de evidencia de las pruebas. `attribution/source_changes.json` vincula las huellas anteriores y nuevas; `tools/verify_attribution.py` comprueba la correspondencia reversible. El README de la compilación original se conserva en `evidence/BUILD_README.md`. El README actual y `LICENSING.md` explican este alcance; se regenera `SHA256SUMS.txt`. Cli conserva MIT y los otros textos de dependencias mantienen Apache-2.0.

Los documentos en inglés y español comparten las mismas declaraciones, enunciados matemáticos, alcance y limitaciones. Utilizan la plantilla LaTeX local de Paper I. La compilación documental y la inspección de páginas renderizadas son distintas de la inspección de la evidencia registrada de la prueba Lean.

## 9. Atribución y preparación del lanzamiento

El teorema de Nash-Williams y los resultados fraccionales formalizados aquí se atribuyen a Delcourt y Postle [3]. La matemática que los sustituye en los puntos señalados se explica en los dos papers. El autor es responsable de comprobar que los enunciados escritos coincidan con sus versiones formales. Los métodos y límites de estas sustituciones deben permanecer visibles en los manuscritos, no solamente aquí.

Claude, de Anthropic, y ChatGPT/Codex, de OpenAI, se utilizaron para explorar y comprobar argumentos y para preparar el manuscrito. Aristotle, de Harmonic, contribuyó a la exploración de enfoques, la búsqueda de contraejemplos, el desarrollo de demostraciones y su formalización y revisión en Lean. Su función no se limitó a traducir pruebas ya terminadas.

Certo [5] y Jacobian [6] se utilizaron para explorar casos finitos, comprobar identidades y evaluar construcciones candidatas. Estos cálculos orientaron el desarrollo de los argumentos; las comprobaciones finitas no se trataron como pruebas de enunciados universales.

El autor es responsable de los argumentos, las citas, el código y la exposición. Ningún sistema de IA figura como autor.

Las fuentes alineadas y la evidencia seleccionadas están en el archivo del commit `afa9f4a06b0e`. Esta referencia requiere acceso al repositorio y no afirma disponibilidad pública. El archivo anterior se conserva sin cambios como registro histórico; el manuscrito y el documento complementario actuales son archivos separados.

Paper II preprint 1.0 [2] presenta el refinador por enlaces de tres pasos y el ensamblaje cuantitativo en el Apéndice B, el teorema semidenso en el Apéndice C y las construcciones PAR/BAL en el Apéndice E. El Apéndice H aporta pruebas escritas de la biblioteca compatible y de las construcciones auxiliares generales; los certificados finitos de familias siguen siendo datos explícitos de las fuentes. Los dossiers anteriores se conservan en [7]. Paper I utiliza ahora las fuentes alineadas para sus entradas tipadas seleccionadas, comprobadas dentro de su propia clausura limpia. Esto no extiende su validación a todo Paper II.

La evidencia formal seleccionada sigue siendo la compilación limpia local completa histórica. La revisión de cabeceras se verifica mediante comparación de bytes y reconstrucción reversible de

las fuentes; no es una compilación de las huellas corregidas ni una reproducción independiente. Se incluyen las licencias de fuentes y dependencias. Este documento y su manuscrito permanecen privados hasta aprobar el lanzamiento coordinado.

## Apéndice. Declaraciones principales en Lean

Los teoremas fraccionales, el teorema de emparejamientos y los resultados de descomposición tipada se desarrollan por separado. El ensamblaje final los importa mediante los enunciados utilizados en la división en casos. Esto permite comparar las hipótesis de cada resultado con las propiedades demostradas para el grafo residual; las Secciones 4–7 documentan sus archivos de origen y dependencias.

El índice incluye el espacio de nombres cuando el nombre corto sería ambiguo; las Secciones 4–7 proporcionan los archivos, los nombres completos de las demás declaraciones y los registros de dependencias.

| Resultado | Declaración |
|---|---|
| Teorema principal (Delcourt–Postle 1.8) | `nash_williams_via_paperII` |
| Delcourt–Postle 1.4 | `NashWilliams.Frac.exists_fractional_weights` |
| Delcourt–Postle 1.6 | `NashWilliams.Frac.theorem_1_6_proof` |
| Delcourt–Postle 2.6 | `NashWilliams18.theorem_2_6` |
| Ehard–Glock–Joos 1.2 | `Pseudorandom.theorem_1_2` |
| Proposición 3.1 | `NashWilliams18.PaperII.margin_four_colour_decomposition` |
| Teorema 3.2 | `idealized_F12` |
| Teorema 3.3, tal como se utiliza aquí | `theorem146_F0` |
| Proposición 4.1 | `regular_seed_nibble` |
| Proposición 4.2 | `roundUnivSpec124_holds` |
| Corolario 4.3 | `branch124_of_round` (condicional a `RoundSpec124`; instanciado más abajo) |
| Delcourt–Postle 11.4 | `Sampling.maxcut_sample` |
| Delcourt–Postle 14.4 | `lemma_14_4_configs_proof` |
| Lema A.1 | `ParityFill.signed_certificate` (versión más fuerte con pequeño defecto de grado) |
| Paper II, Lema SC (triángulos ordinarios) | `NashWilliams18.absorber_onePart` |

La formalización independiente de EGJ está registrada en Palomar como PALOMAR-2026-10-17-000004, versión 1 [4]. Abarca los Teoremas 1.1, 1.2, 1.3, 3.1 y 4.1 del artículo de EGJ identificado en ese registro; la prueba de Nash–Williams utiliza el Teorema 1.2. Este registro corresponde al paquete independiente, no a la validación conjunta del presente desarrollo.

Para el Corolario 4.3, el ensamblaje pasa `roundUnivSpec124_holds` por `roundSpec124_of_univ` a `branch124_of_round`; no queda una hipótesis de ronda de cobertura en la declaración principal. Este apéndice es un índice, no el registro de reproducción. Las versiones del entorno, las fuentes exactas, los informes de axiomas y el procedimiento de compilación desde cero corresponden a las Secciones 4–7.

## Referencias


[1] J. P. Traverso Gianini, *Triangle decompositions I: Formalizing the Nash-Williams theorem in Lean*, preprint 1.0 del manuscrito, 20 de octubre de 2026. Documento.

[2] J. P. Traverso Gianini, *Triangle decompositions II: Robust absorption for typed triangles*, preprint 1.0 (2026). Documento.

[3] M. Delcourt y L. Postle, *A Proof of Nash-Williams' Conjecture*, arXiv:2606.11178v1 (2026). Identificación bibliográfica tomada de Paper I.

[4] Palomar Registry, *PALOMAR-2026-10-17-000004*, version 1 (2026). https://palomar-registry.org/entry.html?id=PALOMAR-2026-10-17-000004&version=1

[5] J. P. Traverso Gianini, *Certo*, repositorio de software. https://github.com/jtraverso/certo-math.

[6] *Jacobian*, repositorio de software. https://github.com/morluto/jacobian.

[7] J. P. Traverso Gianini, *Paper II: dossiers de demostraciones escritas*, registro privado de fuentes, commit 61f752fc87bff6f5654725b23ce10ad4302364ab (19 de octubre de 2026). Registro fijo.